

Environment Mapping

M. C. Escher – Self Portrait

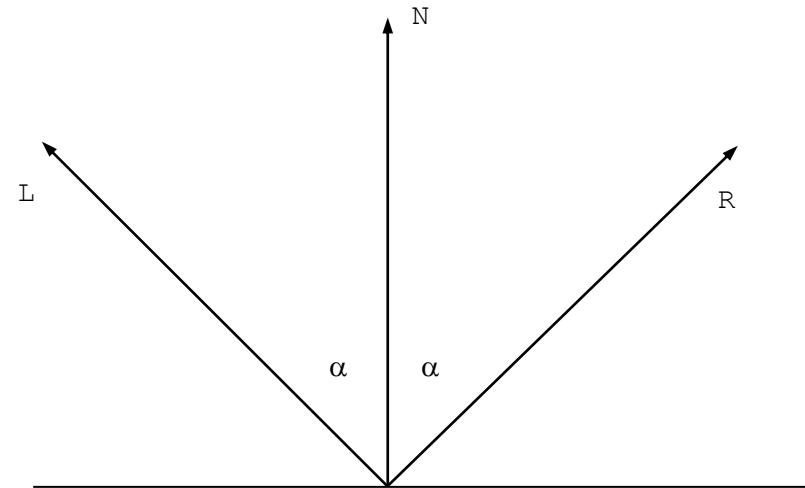


Environment Map / Reflection

- The goal is to render object as if it is perfectly reflective, so that the colors on the object's surface are those reflected to the eye from its surrounding.
- Reflection depends on:
 - Eye position
 - Surface position
 - Surface normal

Reflection Vector

- Where:
 - L is the vector toward the light
 - N is the plane normal
 - R is the reflection vector



Reflection Vector

- R is the reflection vector of L and can be computed as follows:

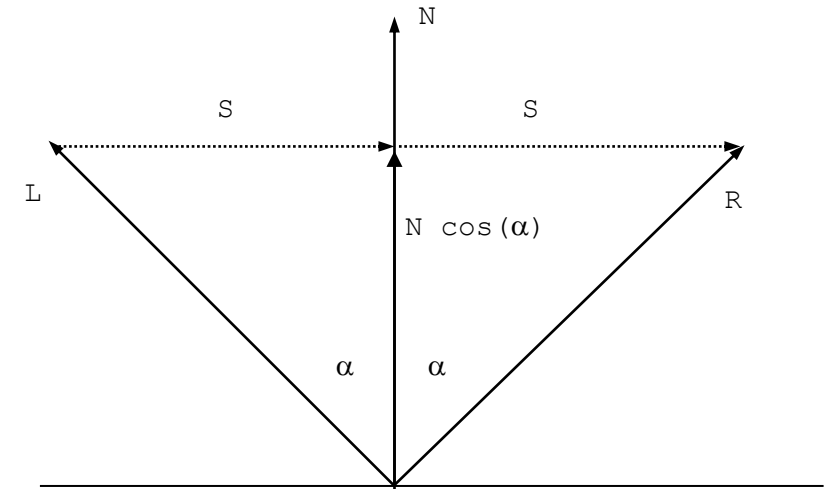
$$R = N \cos(\alpha) + S$$

$$S = N \cos(\alpha) - L$$

Substituting S , we have:

$$R = 2N \cos(\alpha) + L$$

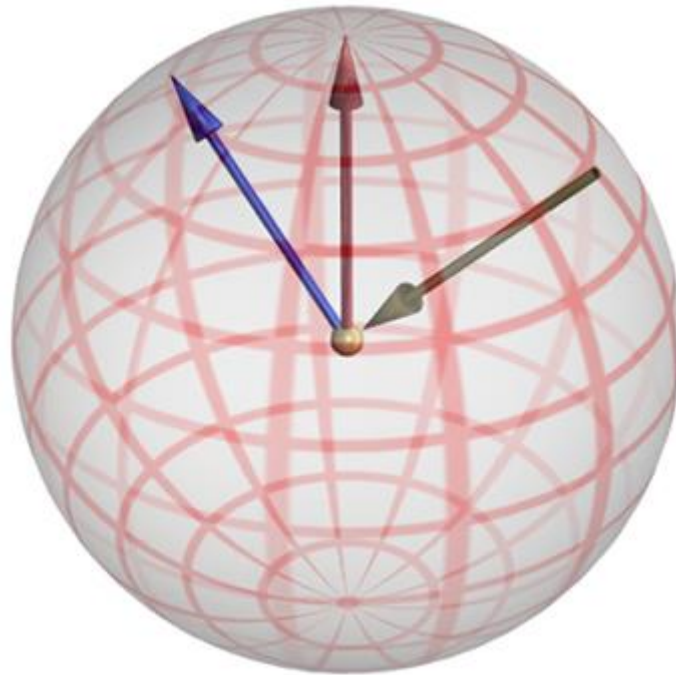
$$R = 2N(N \cdot L) - L$$



Environment Map

- A map used to approximate the environment relative to a reflector.
- The environment map construction assumes that the reflective object is 'small' relative to its environment, i.e. the viewer and other objects being reflected are infinitely far away.

Environment and Object



Environment Map

- General process in environment mapping (in world space):
 - Load (or generate) the image(s) that represent the environment.
 - For each fragment on the reflective object:
 - Calculate the viewing vector
 - Calculate the normal vector
 - Calculate the reflection vector
 - Transform the reflection vector to texture coordinate
 - Use the texture coordinate to get the texel in the environment map

Environment Map

- An environment reflection needs position and direction
 - Position from renderer
 - Direction from normal and view direction

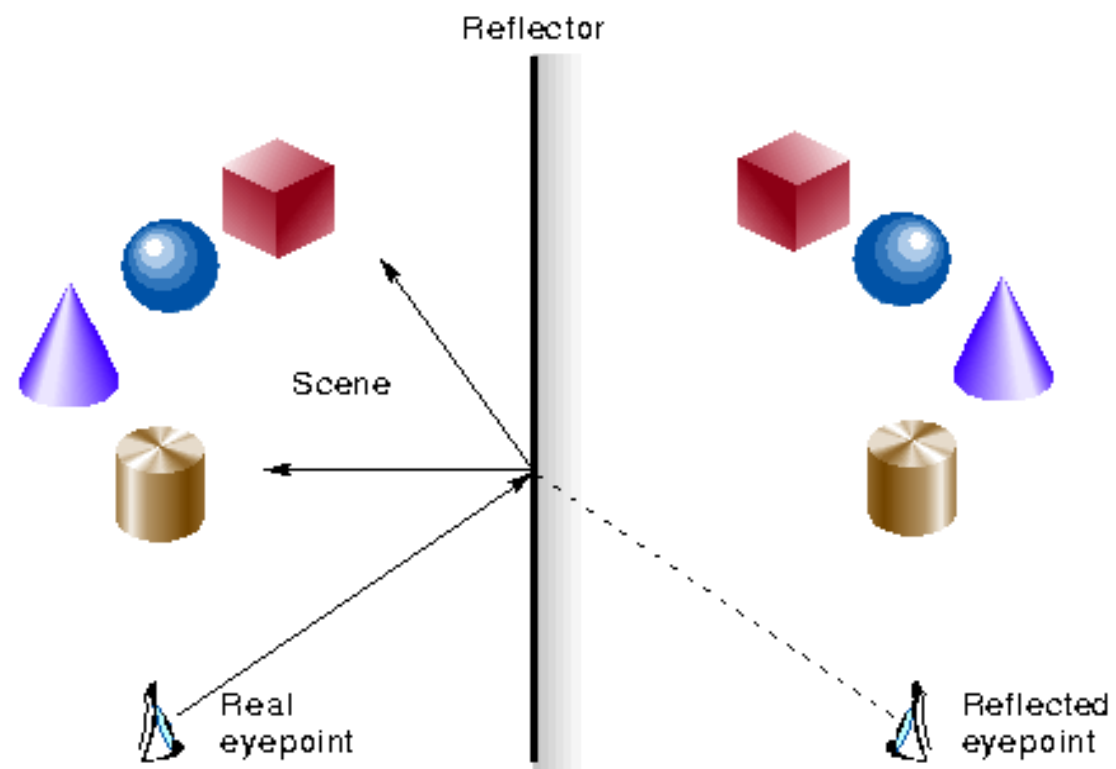
Environment Map

- Common environment map types:
 - Planar map
 - Polar spherical map
 - Spherical map
 - Cube maps
 - Dual parabolic maps
- All types can be loaded or generated in runtime

Planar Environment Map

- Used mainly for a planar reflector (mirror, calm water, etc).
- Process to generate the texture map:
 - Mirror the scene along the reflector plane
 - Render the scene from the new camera position to generate the environment map

Illustration



Planar Environment Map

- To calculate the texture coordinate:

$$s = \frac{V_x}{V_w} * 0.5 + 0.5$$
$$t = \frac{V_y}{V_w} * 0.5 + 0.5$$

- Where:
 - V is a position of the vertex (or fragment) in NDC space

Planar Environment Map

- Pro:
 - Easy to generate
 - Only need one map
 - Very good approximation
- Con:
 - Very specific use
 - View dependent. Need to be regenerated for different viewer position or orientation.

Polar Spherical Environment Map



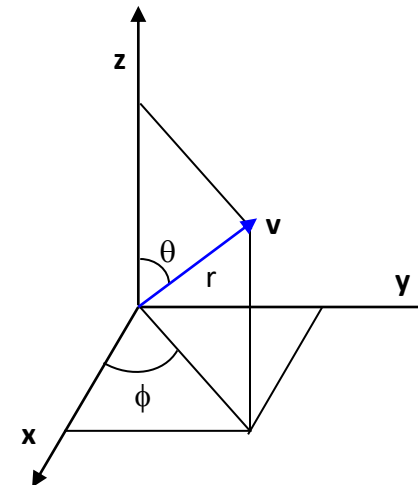
Source: <http://paulbourke.net/miscellaneous/cubemaps/>

Polar Spherical Environment Map

- Environment map is a texture that is wrapped around the sphere. Basically, each texel correspond to a direction in polar coordinate.
- To generate the map, for each (u,v) in the texture coordinate, throw a ray along (ϕ, θ) and calculate its intersection with the environment.

$$\phi = u * 2\pi$$

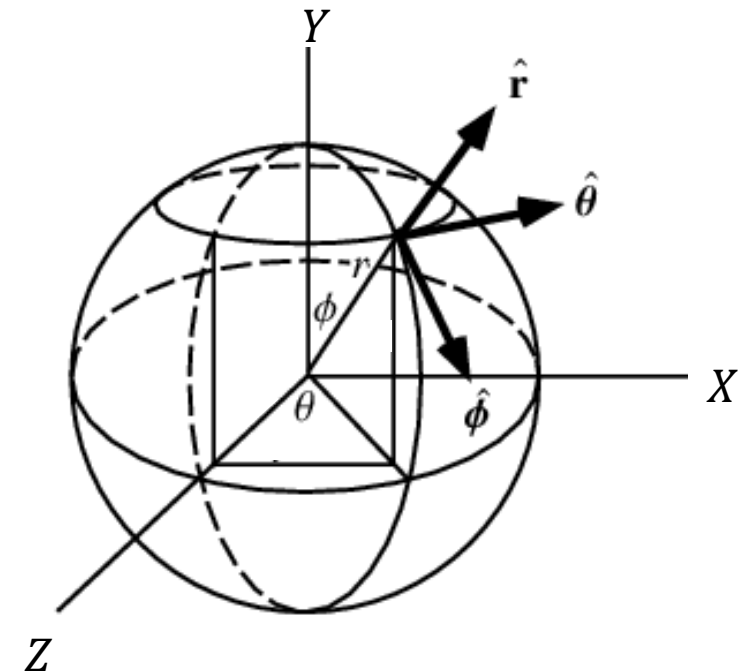
$$\theta = v * \pi$$



Polar Spherical Environment Map

- To calculate the texture coordinate from a reflection vector (in world space), use polar map inverse.
- Assume radius is 1

$$(u, v) = \left(\frac{\tan^{-1} \frac{x}{z}}{2\pi}, \frac{\cos^{-1} y}{\pi} \right)$$



Polar Spherical Environment Map

- Pro:

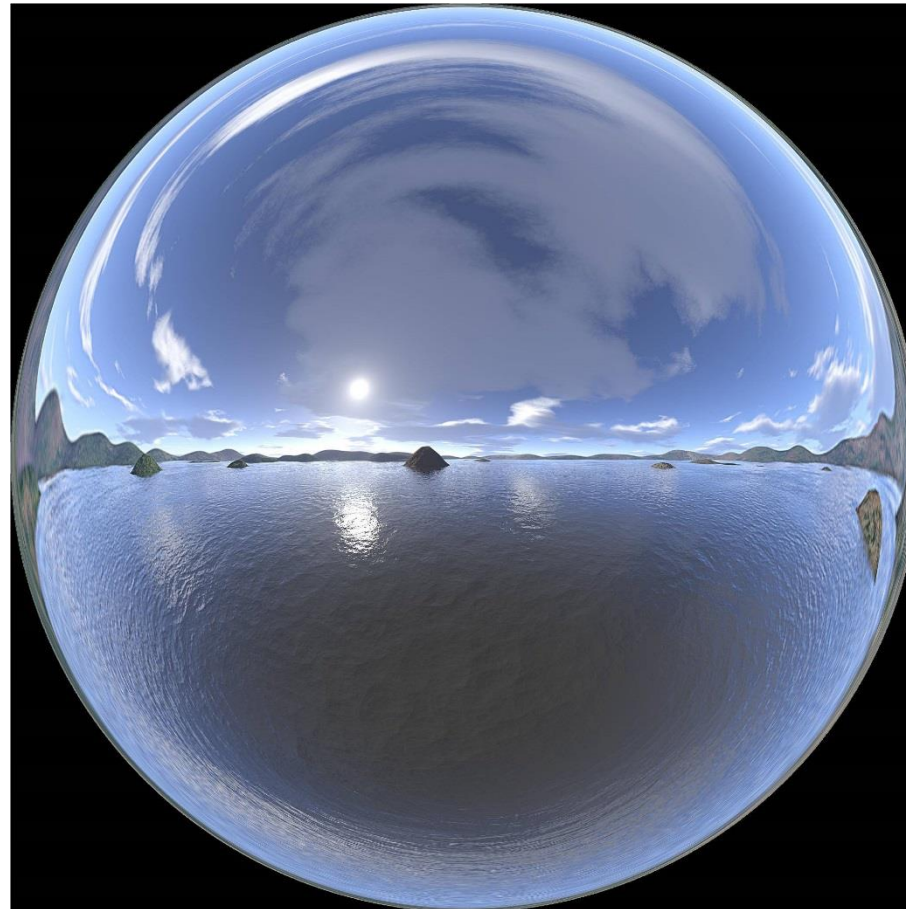
- Simple
- View independent



- Con:

- Distortion is very bad near the poles
- Linear interpolation of texture coords fails when:
 - cross boundary
 - contain poles
- Hard to generate in real-time

Spherical Environment Map



Spherical Environment Map

- The texture is a square texture where only the center circle is used.
- Ways to generate the map:
 - Take a picture of a reflective sphere using a camera with infinite focal point.
 - Generated from cube maps by rendering the cube maps onto a distortion grid.

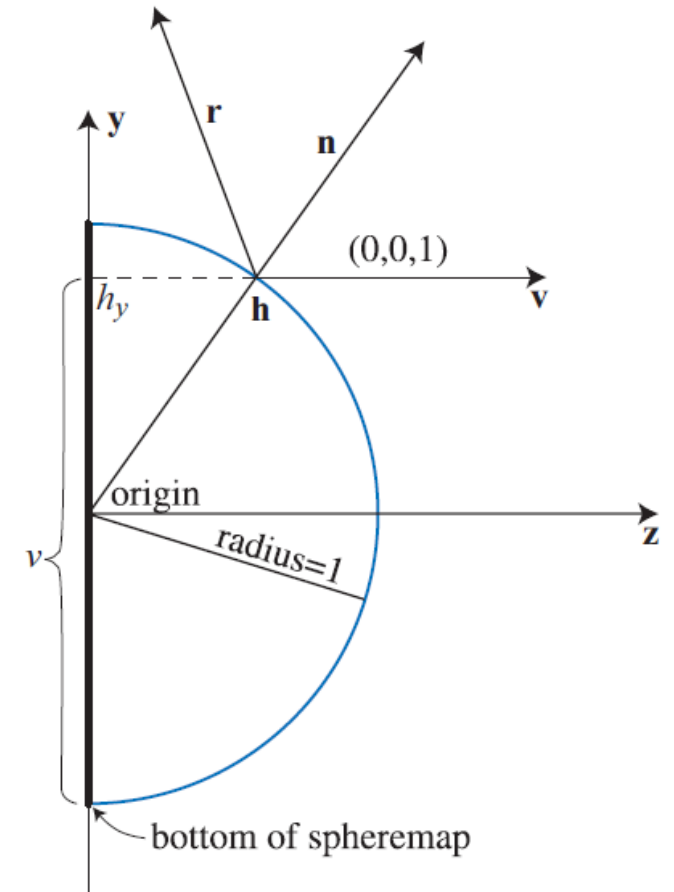


Spherical Environment Map

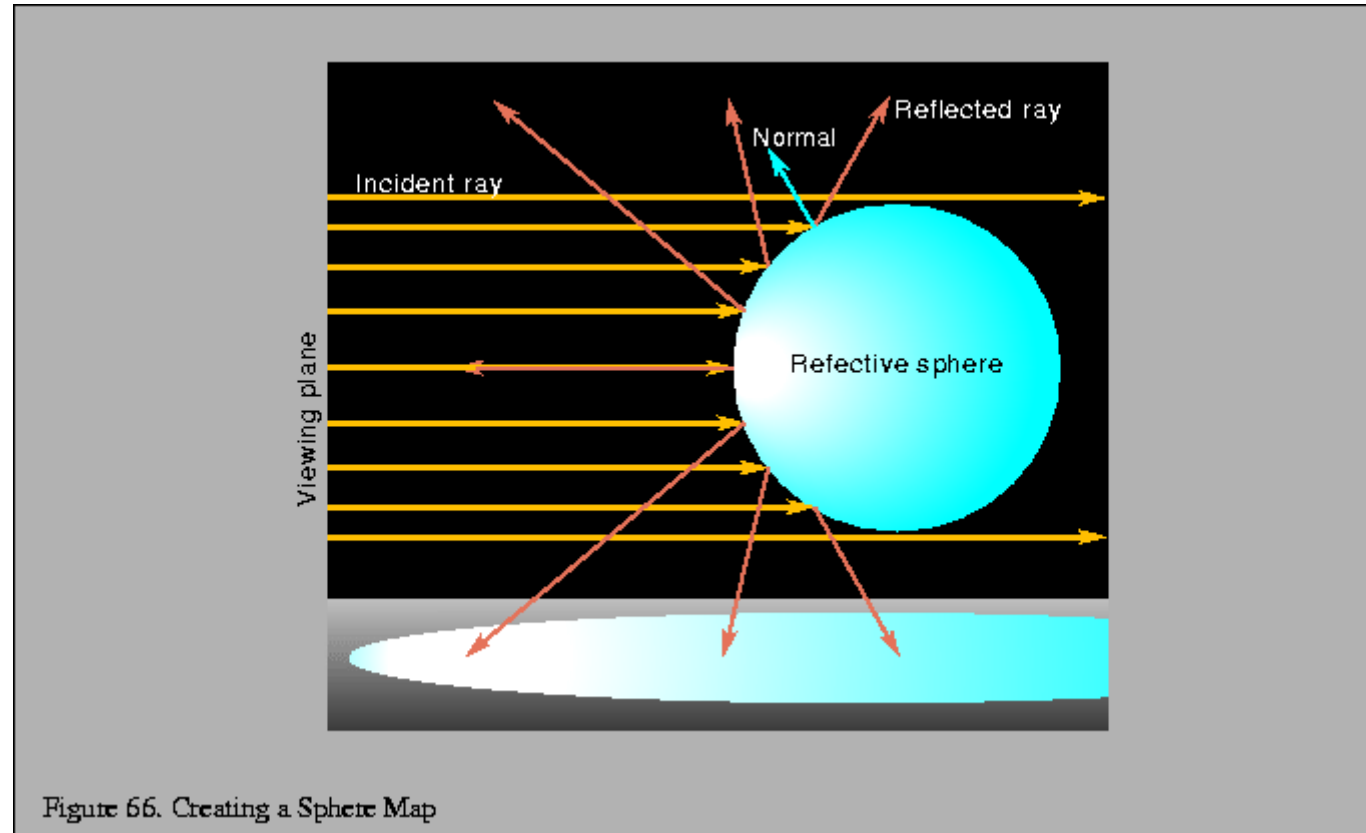
- To calculate the texture coordinate from a reflection vector (R_x, R_y, R_z):
 - View vector $V = (0, 0, 1)$
 - The normal is halfway between R and V

$$N = \frac{R + V}{|R + V|}$$

- The normal correspond to a position P on the sphere and $P = N$ since the sphere's radius = 1.
- So, can use $(P_x, P_y) = (N_x, N_y)$ as (u', v') .
- The calculated (u', v') need to be mapped between $[0, 1]$ to calculate the final (u, v) .



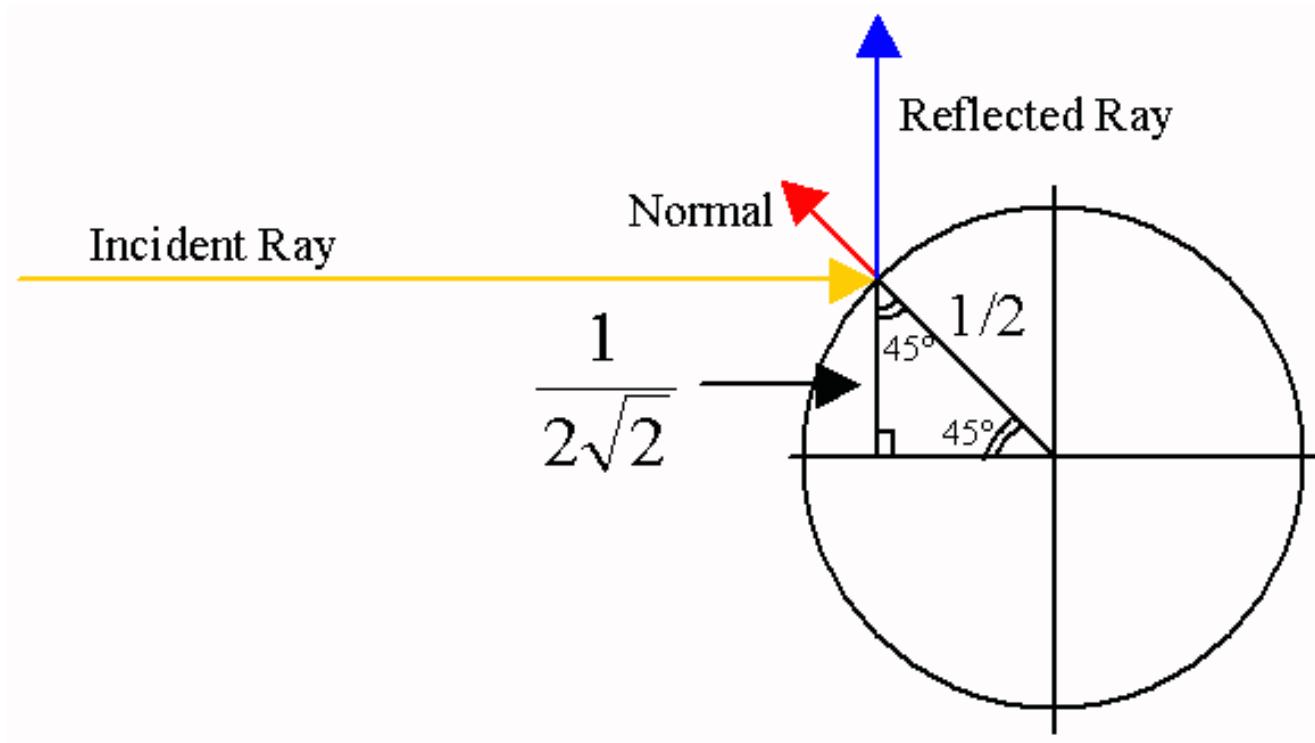
Calculating the Reflection Vector



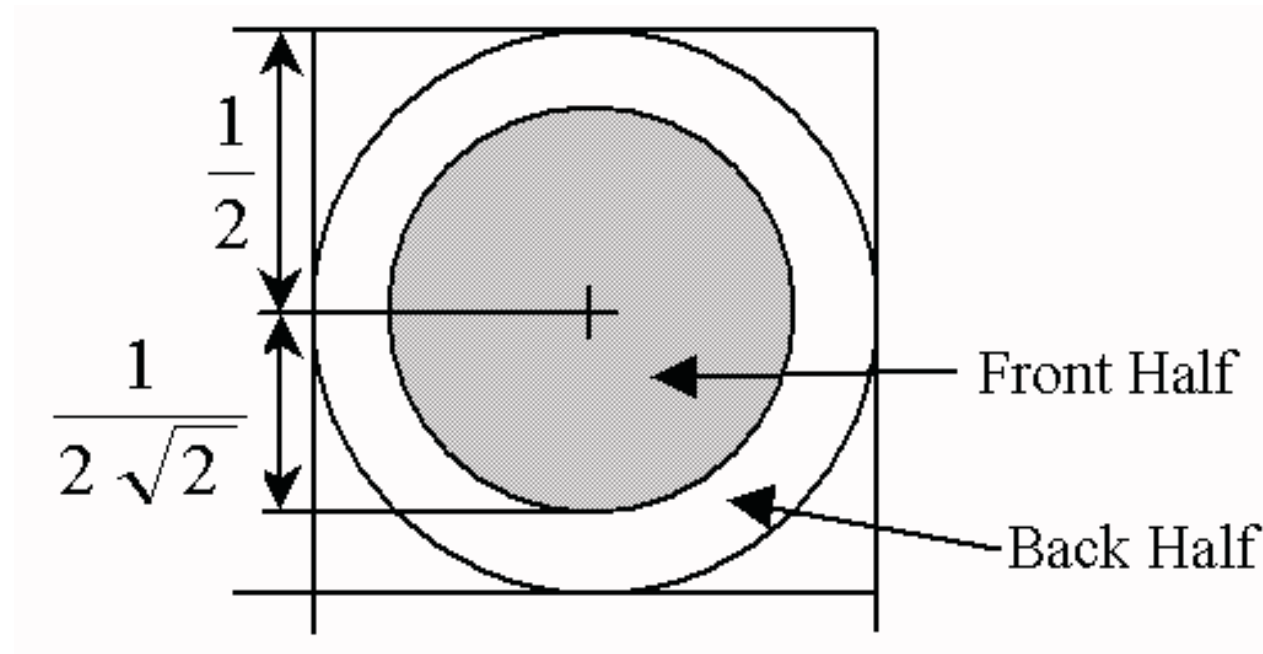
Spherical Environment Map

- Pro:
 - Can be generated with a camera or programmatically followed by distortion calculation
 - Needs one image
- Con:
 - View dependent
 - Interpolation of the texture coordinates also leads to artifact
 - Hard to generate in real-time
 - Low resolution on sphere edges.

Limitations of Spherical Mapping

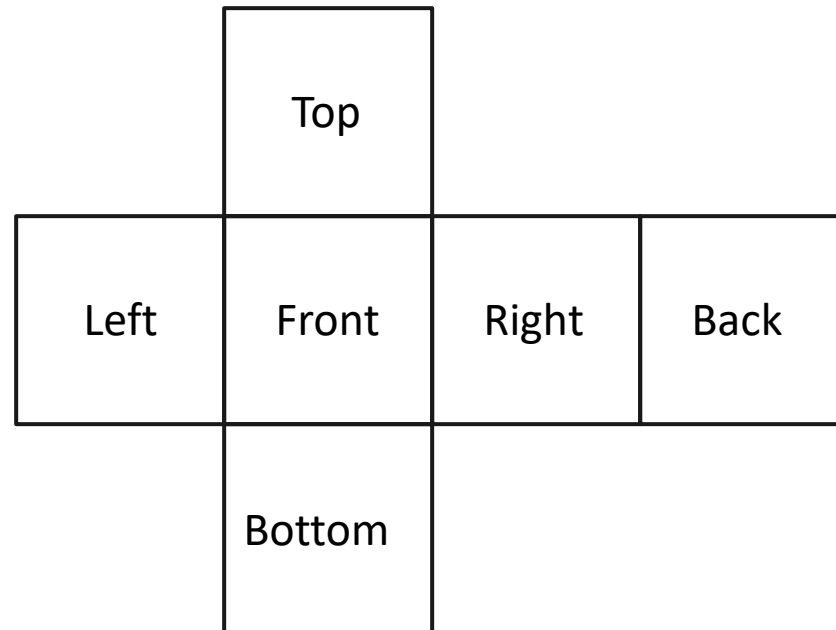
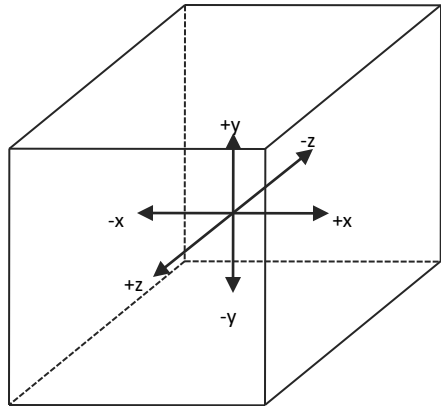


Limited Resolution for “back” texels

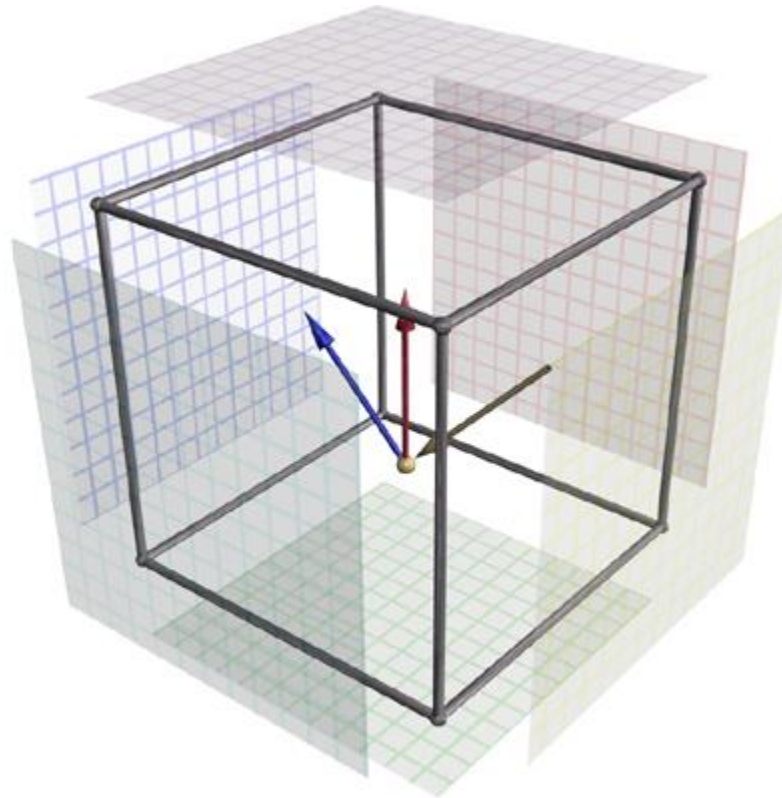


Cubic Environment Maps

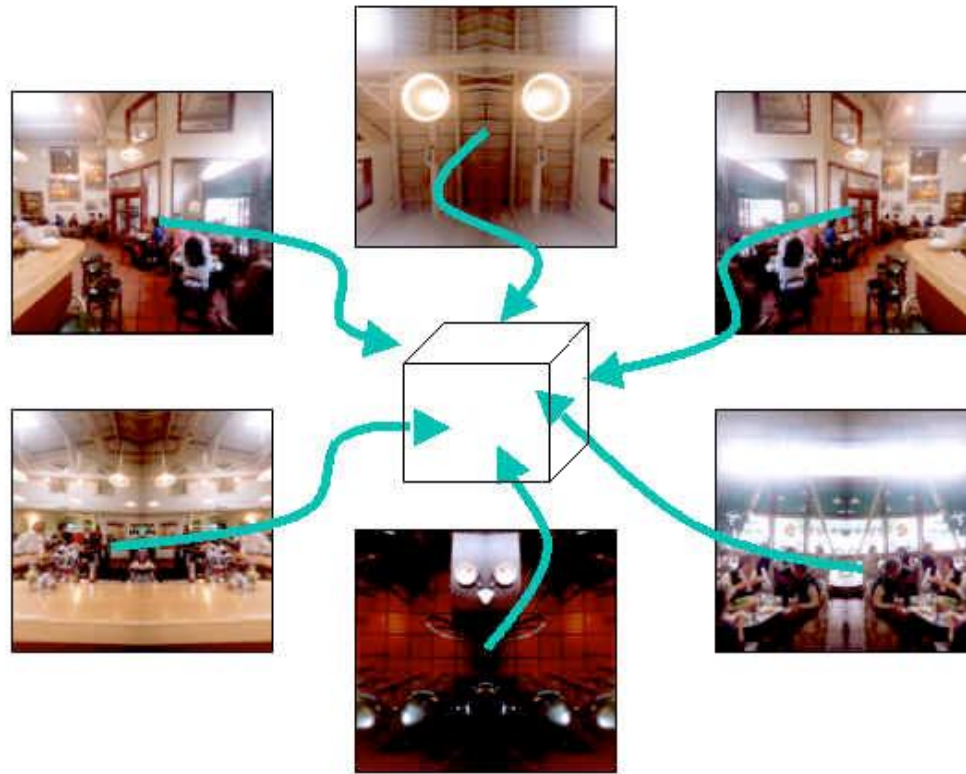
- The environment maps are 6 square texture maps. One for each axis aligned directions.



Environment and Object



Cubic Environment Maps



Spherical Map

Cubic Environment Maps

- Process to generate the maps:
 - Position the camera at the reflective object position.
 - Set the FOV to 90 degree and aspect ratio to 1.
 - For each map:
 - Orient the camera to point along the associated axis.
 - Render the scene

Cubic Environment Maps

- To calculate the texture coordinate from R :
 - Choose the texture map based on the greatest magnitude of R component.
 - Intersect R with the cube face

$$R' = \frac{R}{\max(|R_x|, |R_y|, |R_z|)}$$

- Use the other 2 component that are not the greatest magnitude as the texture coordinate.

Cubic Environment Maps

- To calculate the texture coordinate from $R(x, y, z)$:

$$R' = \frac{R}{\max(|x|, |y|, |z|)}$$

- Use the other 2 component that are not the greatest magnitude as the texture coordinate.

x greatest:

When $x < 0$

$$uv = \frac{(z, y)}{|x|}$$

Else

$$uv = \frac{(-z, y)}{|x|}$$

y greatest:

When $y < 0$

$$uv = \frac{(x, -z)}{|y|}$$

Else

$$uv = \frac{(x, z)}{|y|}$$

z greatest:

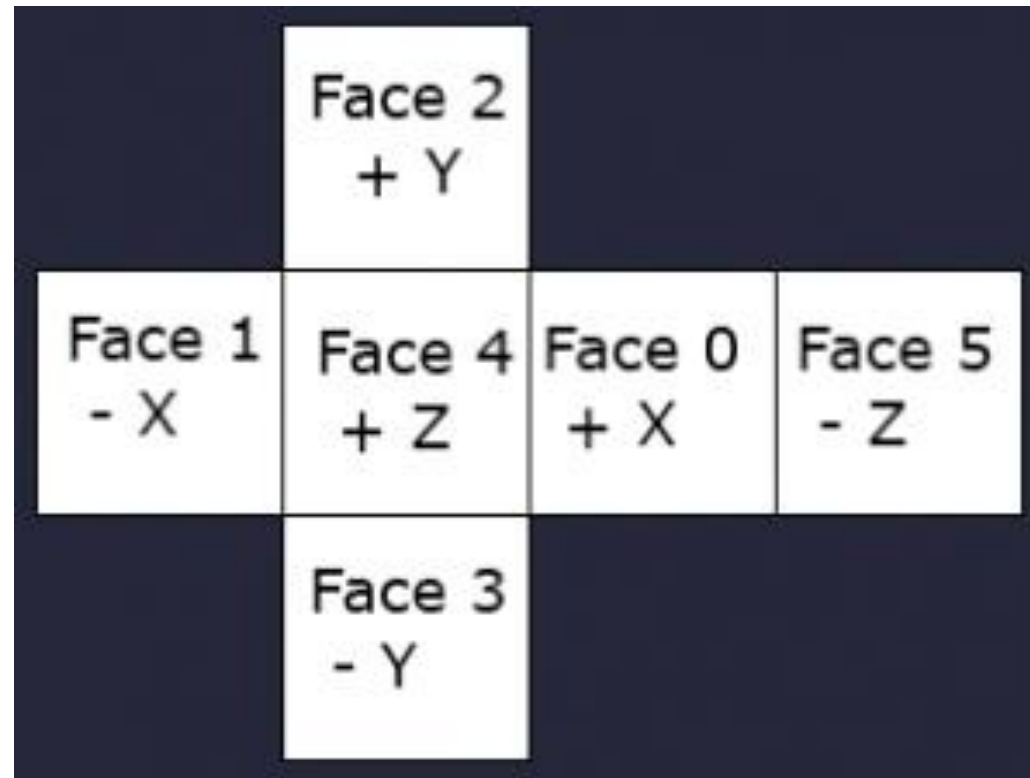
When $z < 0$

$$uv = \frac{(-x, -y)}{|z|}$$

Else

$$uv = \frac{(x, -y)}{|z|}$$

In pictures ...

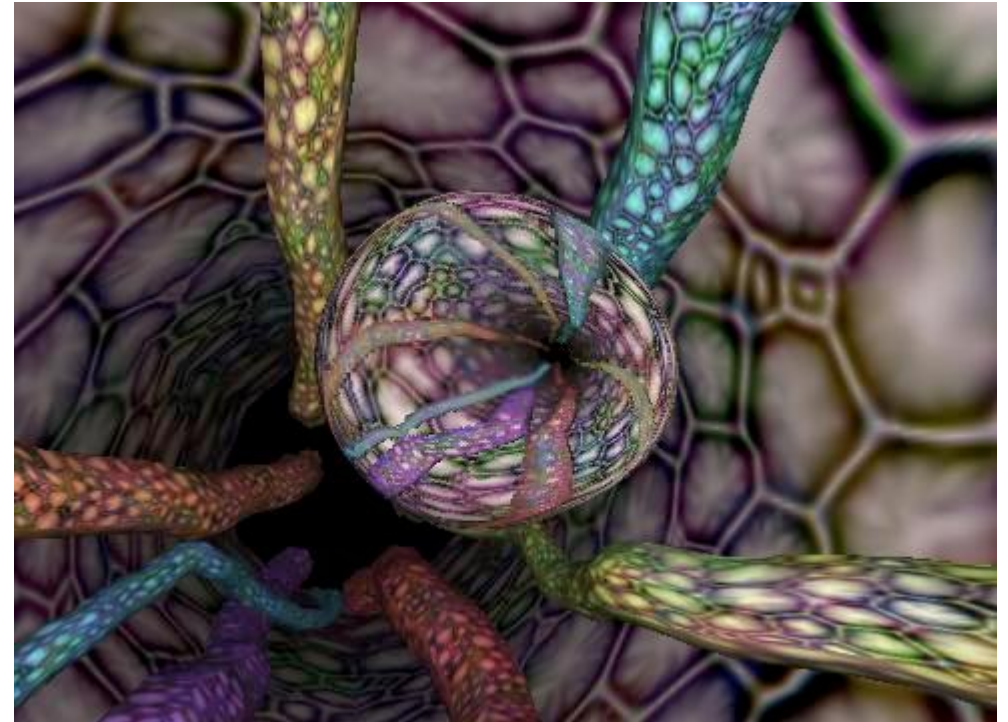


Cubic Environment Mapping

Cube Map



Result of rendering

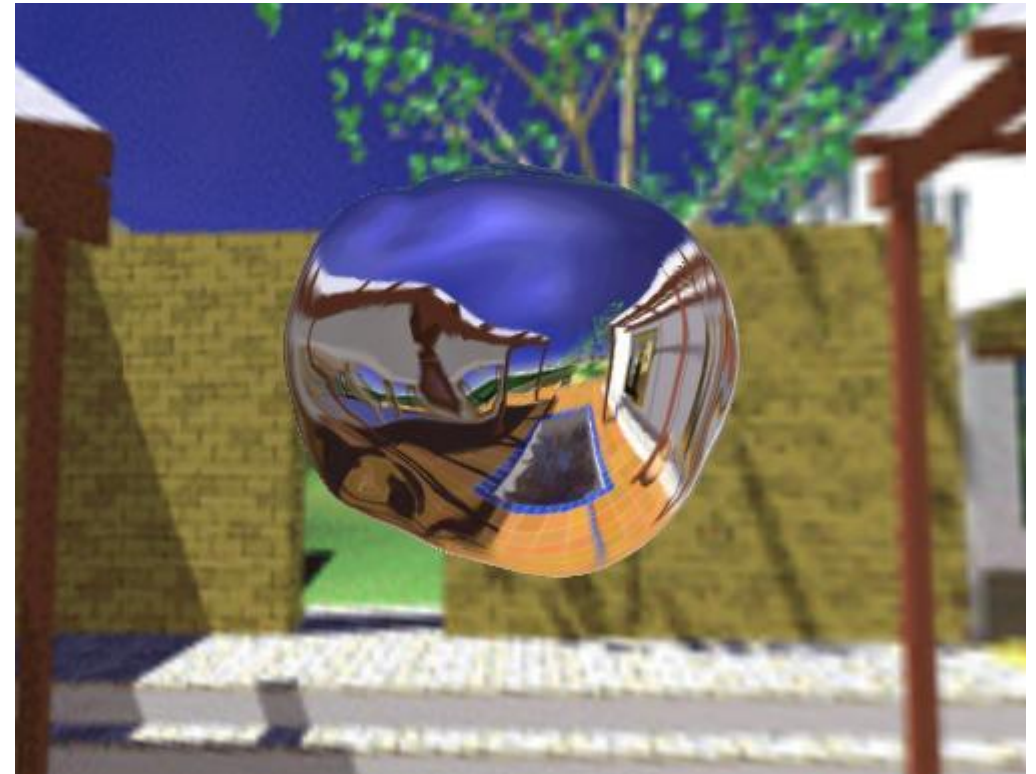


Cubic Environment Mapping

Cube Map



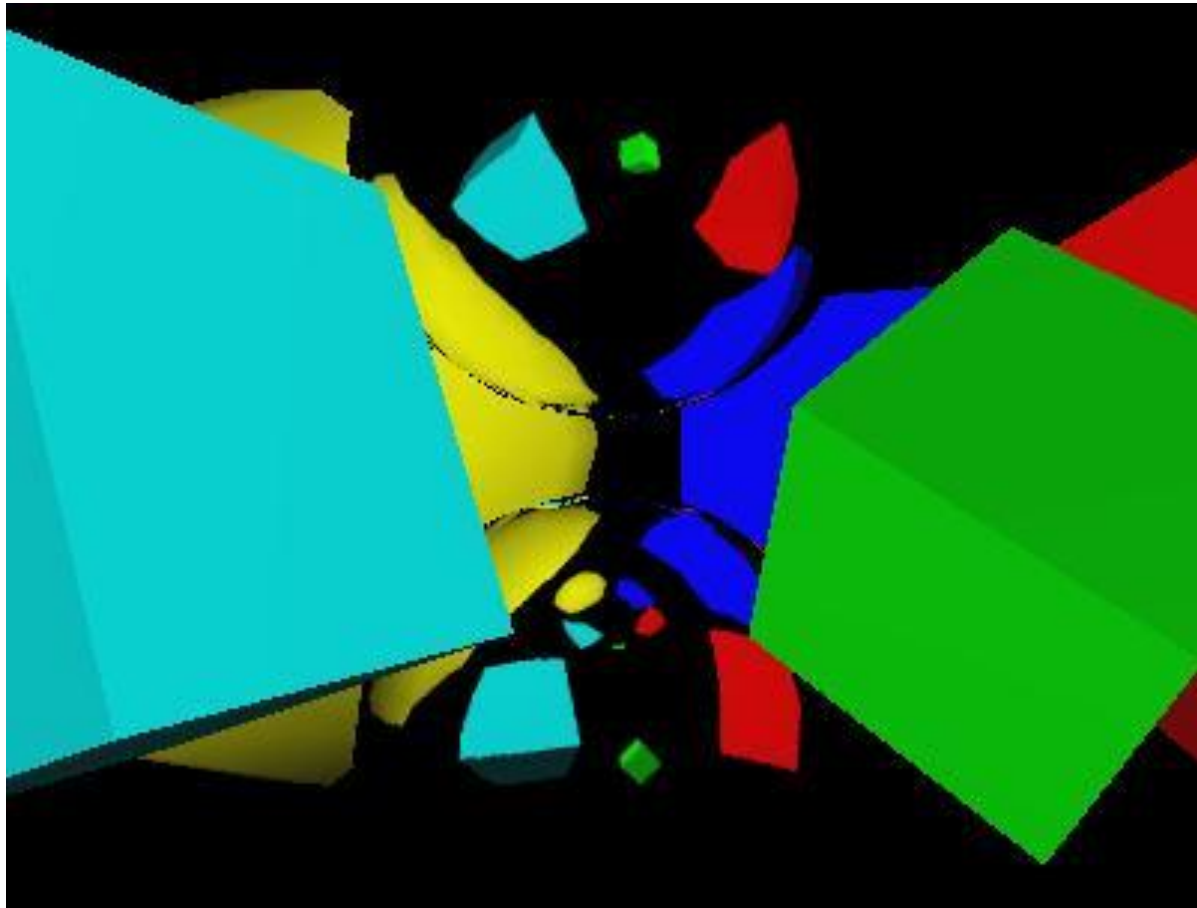
Result of rendering



Cubic Environment Maps

- Pro:
 - Little distortion
 - No pole/boundary interpolation problems
 - Can be generated programmatically
 - View independent
- Con:
 - Needs six images

Recursive Reflection!



18 passes!

Dual Parabolic Mapping

- Encodes the environment in two texture maps to retain resolution for view independence.
 - One for the front side
 - One for the back side

- Parabolic equations:

- Front side:

$$f(x, y) = 0.5 - 0.5 * (x^2 + y^2)$$

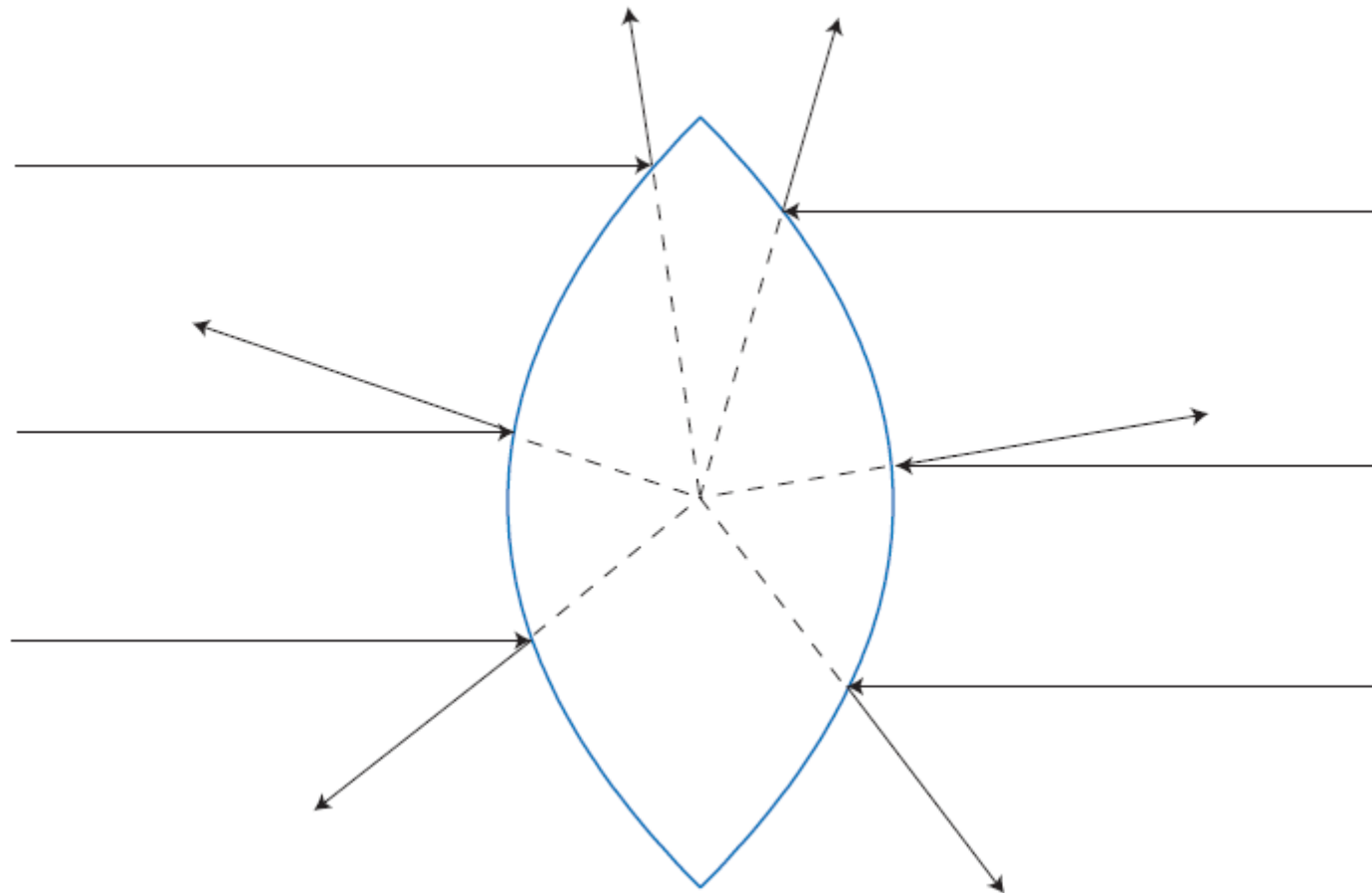
- Back side:

$$f(x, y) = -0.5 + 0.5 * (x^2 + y^2)$$

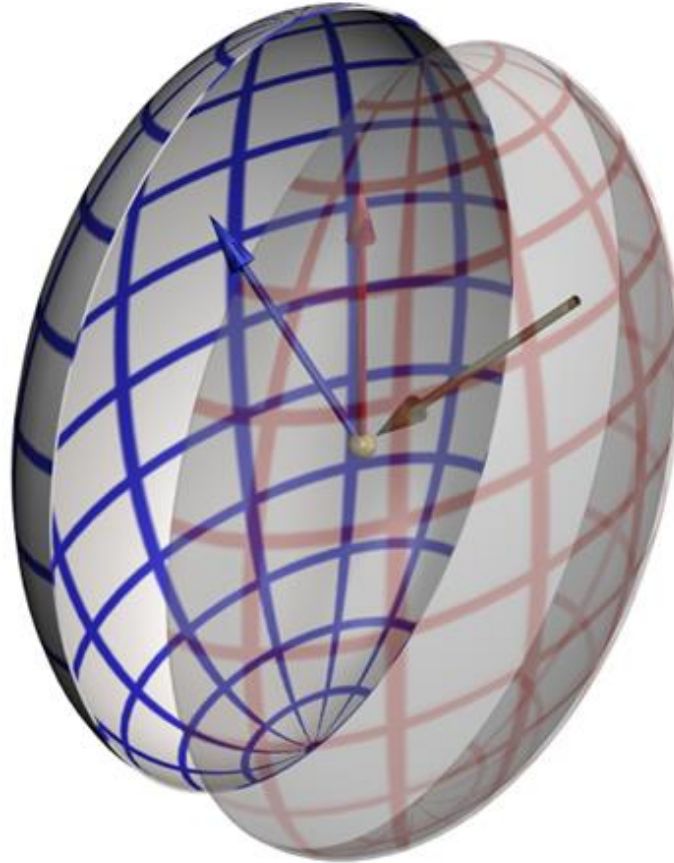
- For both sides:

$$x^2 + y^2 \leq 1$$

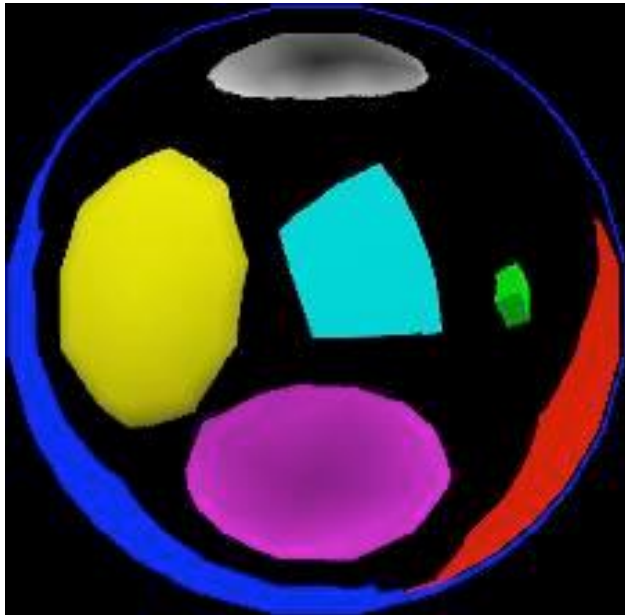
Dual Paraboloid Mapping



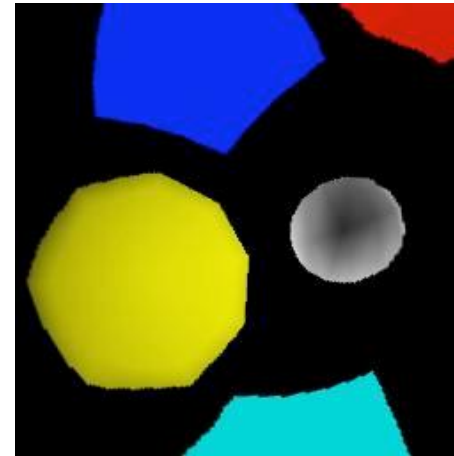
Environment and Object



Comparing Sphere Map and Parabolic Maps



Sphere map



front



back

Dual Parabolic Mapping

- Process to generate the parabolic map
 - Position the camera at the reflector position
 - For each side to be rendered:
 - Orient the camera
 - For each object in the scene
 - In vertex shader, distort the vertex position to its corresponding position on the texture map.
 - Render the object to the texture.

Dual Parabolic Mapping

- Calculating the texture coordinate from a reflection vector R:
 - Based on R_z choose the texture map
 - For the front one:

$$u = \frac{R_x}{R_z + 1} * 0.5 + 0.5$$
$$v = \frac{R_y}{R_z + 1} * 0.5 + 0.5$$

- For the back one:

$$u = \frac{R_x}{R_z - 1} * 0.5 + 0.5$$
$$v = \frac{R_y}{R_z - 1} * 0.5 + 0.5$$

Dual Parabolic Mapping

- Pro:
 - View independent.
 - Can be generated programmatically
- Con:
 - Need two texture maps

References

- <https://www.opengl.org/archives/resources/code/samples/sig99/advanced99/notes/node174.html>
- <https://www.opengl.org/archives/resources/code/samples/sig99/advanced99/notes/node176.html>
- <https://www.opengl.org/archives/resources/code/samples/sig99/advanced99/notes/node184.html>
- <http://developer.nvidia.com/cube-map-ogl-tutorial>
- <http://www.unc.edu/~zimmons/cs238/maps/environment.html>